

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M Madhu Sneha	Reg. No.:	192425092
Section / Batch:	AI&ML	Date:	

Code of Conduct

I M. Sneha/192425092 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sneha

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	6	5	5	6	6	28	80
2	5	6	5	6	5	27	
3	5	5	5	5	5	25	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Case Study 1: E-Commerce Smart Search and Product Prediction System

Answer 1: MLE of the Bigram Probability $P(\text{learning} \mid \text{machine})$

Formula: $P(w_2 \mid w_1) = C(w_1, w_2) / C(w_1)$

Given: $C(\text{machine}) = 3$, $C(\text{machine learning}) = 3$

$P(\text{learning} \mid \text{machine}) = C(\text{machine, learning}) / C(\text{machine}) = 3 / 3 = 1.0$

Interpretation: A probability of 1.0 means that in the training corpus every occurrence of the word “machine” is immediately followed by “learning”. This strong association allows the e-commerce search system to predict “learning” with full confidence as soon as a customer types “machine”, which directly improves autocomplete relevance and speed for this bigram. However, because the estimate comes from a very small corpus, such an extreme probability is a sign of overfitting to limited data; in a production system this value would normally be smoothed so that other, unseen but grammatically valid continuations are not assigned zero probability.

Answer 2: Backoff Model for the unseen sequence “machine learning transforms”

Step 1 (Trigram level): $C(\text{learning, transforms}) = 0$, so the trigram “learning transforms” is unseen and the model must back off to a lower order.

Step 2 (Bigram level): $C(\text{learning, transforms})$ is also 0 at the bigram level, so the model backs off further.

Step 3 (Unigram level): “transforms” does not occur anywhere in the corpus vocabulary (machine, learning, improves, business, enables, automation, drives, innovation), so it is an out-of-vocabulary (OOV) word with $C(\text{transforms}) = 0$.

Because even the unigram count is zero, pure backoff must be combined with a smoothing technique that reserves a small non-zero probability mass for unseen words. Using Add-One (Laplace) smoothing:

$$P(\text{transforms}) \approx (C(\text{transforms}) + 1) / (N + V) = 1 / (12 + 8) = 1 / 20 = 0.05$$

where $N = 12$ is the total token count of the corpus and $V = 8$ is the vocabulary size.

Why backoff is useful: Customers routinely type novel word combinations that never appeared in the training data — this is the well-known problem of data sparsity. Under the plain chain rule, a single zero-count n-gram anywhere in a sequence would force the whole sequence probability to zero. Backing off to a lower, more reliably estimated order (trigram $\rightarrow$ bigram $\rightarrow$ unigram, with

discounting/smoothing at the base) lets the system still return a small but non-zero, ranked probability, so autocomplete keeps functioning gracefully instead of failing outright on unseen queries.

Answer 3: Deleted Interpolation for “machine learning improves”

Deleted interpolation formula:

$$P_{\text{interp}}(w_3 | w_1, w_2) = \lambda_1 \cdot P(w_3 | w_1, w_2) + \lambda_2 \cdot P(w_3 | w_2) + \lambda_3 \cdot P(w_3)$$

With $\lambda_1 = 0.5$ (trigram), $\lambda_2 = 0.3$ (bigram), $\lambda_3 = 0.2$ (unigram):

- Trigram MLE: $P(\text{improves} | \text{machine, learning}) = C(\text{machine learning improves}) / C(\text{machine learning}) = 1/3 = 0.3333$
- Bigram MLE: $P(\text{improves} | \text{learning}) = C(\text{learning improves}) / C(\text{learning}) = 1/3 = 0.3333$
[$C(\text{learning}) = 1+1+1 = 3$]
- Unigram MLE: $P(\text{improves}) = C(\text{improves}) / N = 1/12 = 0.0833$ [corpus has $N = 12$ tokens]

$$P_{\text{interp}}(\text{improves} | \text{machine, learning}) = 0.5(0.3333) + 0.3(0.3333) + 0.2(0.0833)$$

$$= 0.1667 + 0.1000 + 0.0167 = 0.2833$$

Sequence probability by the chain rule:

$$P(\text{machine learning improves}) = P(\text{machine}) \times P(\text{learning} | \text{machine}) \times$$

$$P_{\text{interp}}(\text{improves} | \text{machine, learning})$$

$$= (3/12) \times 1.0 \times 0.2833 = 0.25 \times 1.0 \times 0.2833 = 0.0708$$

Interpretation: Combining trigram, bigram, and unigram estimates makes the probability estimate far more robust than relying on the trigram alone. The trigram model is the most context-specific but is also the most fragile when counts are as low as 1, since a single sentence entirely determines its value. By blending it with the more stably estimated bigram and unigram distributions using fixed interpolation weights, deleted interpolation smooths out the high variance of sparse higher-order counts and prevents the model from overfitting to accidental co-occurrences in a small training corpus.

Answer 4: Entropy of the Prediction Distribution

$$\text{Formula: } H(X) = -\sum p(x) \log_2 p(x)$$

Given $P(\text{improves}) = P(\text{enables}) = P(\text{drives}) = 0.33$ (uniform distribution over 3 outcomes):

$$H = -3 \times (0.33 \times \log_2 0.33) = -3 \times (0.33 \times (-1.5995)) = -3 \times (-0.5278) = 1.5834 \approx 1.585 \text{ bits}$$

(This matches the theoretical maximum entropy for 3 equally likely outcomes, $\log_2 3 = 1.585$ bits.)

Interpretation: An entropy of 1.585 bits is the maximum possible uncertainty for a 3-way choice, meaning the model has no informative preference among “improves”, “enables”, and “drives” after

seeing “learning”. High entropy signals that the current bigram context alone is insufficient to confidently predict a single next word. For the search system, this suggests that the UI should present all three completions as ranked suggestions rather than auto-selecting one, and that additional context (e.g., trigram history, personalization signals, or more training data) is needed to reduce uncertainty and sharpen the prediction.

Answer 5: Python Program — N-gram Modeling, Backoff, Interpolation and Entropy

```
import nltk
from nltk import bigrams, trigrams
import numpy as np
import math
from collections import Counter

# -----
# 1. Corpus and Preprocessing
# -----
corpus = ("machine learning improves business machine learning enables "
          "automation machine learning drives innovation")
tokens = corpus.split()

unigram_counts = Counter(tokens)
bigram_counts = Counter(bigrams(tokens))
trigram_counts = Counter(trigrams(tokens))

N = len(tokens)
V = len(set(tokens))

print("Unigram counts:", unigram_counts)
print("Bigram counts:", bigram_counts)
print("Trigram counts:", trigram_counts)

# -----
# 2. MLE Bigram Probability:  $P(\text{learning} \mid \text{machine})$ 
# -----
def mle_bigram(w1, w2):
    return bigram_counts[(w1, w2)] / unigram_counts[w1]

p_learning_given_machine = mle_bigram("machine", "learning")
print(f"\nP(learning|machine) = {p_learning_given_machine:.4f}")

# -----
# 3. Backoff Model: unseen sequence "machine learning transforms"
# -----
def backoff_probability(w1, w2, w3, alpha=0.4):
    if trigram_counts[(w1, w2, w3)] > 0:
        return trigram_counts[(w1, w2, w3)] / bigram_counts[(w1, w2)]
    elif bigram_counts[(w2, w3)] > 0:
        return alpha * (bigram_counts[(w2, w3)] / unigram_counts[w2])
    else:
        # Laplace-smoothed unigram fallback
        return (unigram_counts[w3] + 1) / (N + V)
```



```

p_transforms = backoff_probability("machine", "learning", "transforms")
print(f"Backoff P(transforms | machine, learning) = {p_transforms:.4f}")

# -----
# 4. Deleted Interpolation for "machine learning improves"
# -----
lambda1, lambda2, lambda3 = 0.5, 0.3, 0.2

def interpolated_probability(w1, w2, w3):
    p_tri = trigram_counts[(w1, w2, w3)] / bigram_counts[(w1, w2)] if
bigram_counts[(w1, w2)] else 0
    p_bi = bigram_counts[(w2, w3)] / unigram_counts[w2] if unigram_counts[w2] else
0
    p_uni = unigram_counts[w3] / N
    return lambda1 * p_tri + lambda2 * p_bi + lambda3 * p_uni

p_interp = interpolated_probability("machine", "learning", "improves")
p_sequence = (unigram_counts["machine"] / N) * mle_bigram("machine", "learning") *
p_interp
print(f"\nInterpolated P(improves|machine,learning) = {p_interp:.4f}")
print(f"P(machine learning improves) = {p_sequence:.4f}")

# -----
# 5. Entropy of the next-word prediction distribution
# -----
probs = {"improves": 0.33, "enables": 0.33, "drives": 0.33}
entropy = -sum(p * math.log2(p) for p in probs.values())
print(f"\nEntropy of next-word distribution = {entropy:.4f} bits")

# -----
# 6. Final next-word prediction
# -----
best_word = max(probs, key=probs.get)
print(f"\nFinal predicted next word after 'learning': '{best_word}' "
      f"(probability = {probs[best_word]})")

```

Case Study 2: AI-Powered Hospital Appointment Chatbot

Answer 1: Penn Treebank POS Tagging

Sentence 1: “Book an appointment with the doctor.”

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN ./.

Justification: “Book” occurs in sentence-initial position and functions as an imperative main verb instructing the chatbot to perform an action, followed by the noun phrase “an appointment” acting as its direct object — a purely verbal syntactic slot — so it is tagged VB (base-form verb).

Sentence 2: “The book contains medical information.”

The/DT book/NN contains/VBZ medical/JJ information/NN ./.

Justification: Here “book” is preceded by the determiner “The” and is itself followed by the finite verb “contains” (3rd person singular present), which requires a subject noun phrase before it. “Book” occupies that subject-noun slot, so it is tagged NN.

Answer 2: HMM Likelihood Computation for “book”

$\text{Score}(\text{tag}) = P(\text{Start} \rightarrow \text{tag}) \times P(\text{book} \mid \text{tag})$

$\text{Score}(\text{VB}) = P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB}) = 0.6 \times 0.7 = 0.42$

$\text{Score}(\text{NN}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} \mid \text{NN}) = 0.4 \times 0.3 = 0.12$

Since $0.42 > 0.12$, the most probable tag is VB (Verb).

Why the model favors VB: Both factors point the same way — the emission probability $P(\text{book} \mid \text{VB}) = 0.7$ is higher than $P(\text{book} \mid \text{NN}) = 0.3$, and the prior/transition probability of starting a sentence with a verb, $P(\text{Start} \rightarrow \text{VB}) = 0.6$, is also higher than $P(\text{Start} \rightarrow \text{NN}) = 0.4$. Multiplying these compounds both effects in favor of VB. This mirrors real chatbot usage, where imperative commands such as “Book an appointment”, “Book a table” are extremely common sentence-initial patterns in booking-oriented dialogue, so the HMM has learned a strong sentence-initial verb prior from training data.

Answer 3: Rule-Based vs Stochastic (HMM) POS Tagging

Rule-Based Tagging: Uses handcrafted, deterministic linguistic rules (e.g., “if a word is preceded by a determiner and followed by a finite verb, tag it NN”; “if a word is the first token of the sentence and followed by a determiner+noun phrase, tag it VB”).

- Advantages: transparent and interpretable; requires no training data; easy to audit for a small, closed set of recurring ambiguous patterns typical in a booking domain.
- Limitations: brittle and does not scale to the combinatorial richness of natural language; needs continual manual maintenance as new phrasings appear; conflicting rules are hard to resolve.

Stochastic HMM Tagging: Learns emission and transition probabilities from an annotated training corpus and selects the globally most likely tag sequence (e.g., via the Viterbi algorithm).

- Advantages: generalizes automatically to novel sentences and ambiguous words without hand-authored rules; captures subtle statistical regularities such as the sentence-initial verb bias shown in Answer 2; scales well as more data becomes available.
- Limitations: requires a sufficiently large annotated corpus for reliable probability estimates; behaves as a “black box” relative to rules, making individual decisions harder to explain to a clinician or auditor; can misfire on rare constructions under-represented in training.

For a healthcare chatbot, a hybrid design is often preferable: broad-coverage HMM/statistical tagging, supplemented with a thin layer of rule-based overrides for safety-critical or clinically sensitive phrasings that must never be mistagged.

Answer 4: Standardized Tagsets and Downstream NLP Tasks

A standardized tagset such as the Penn Treebank tagset ensures that every component of the pipeline (tokenizer, POS tagger, parser, intent classifier, entity recognizer, response generator) shares one common label vocabulary, keeping features consistent across models. In a healthcare chatbot this supports:

- Intent detection: verb tags (VB/VBZ/VBP) reliably flag the action a patient wants (“book”, “cancel”, “reschedule”), feeding directly into intent classifiers.
- Entity identification: noun tags (NN/NNP/NNS) help locate candidate entities such as doctor names, departments, or symptoms, improving Named Entity Recognition precision.
- Response generation: knowledge of syntactic roles allows templates or neural generators to produce grammatically correct replies, such as correctly conjugating verbs and ordering noun phrases.

Because the tagset is standardized and well documented, pretrained taggers, parsers, and NLP libraries can be reused off-the-shelf instead of being retrained from scratch for the healthcare domain, reducing engineering effort and improving interoperability with external clinical NLP tools.

Answer 5: Python Program — POS Tagging and HMM Probability Calculation

```
import nltk
nltk.download('punkt')
```

```

nltk.download('averaged_perceptron_tagger')
from nltk import word_tokenize, pos_tag

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

print("Tokenized sentences and Penn Treebank POS tags:")
for s in sentences:
    tokens = word_tokenize(s)
    tags = pos_tag(tokens)
    print(f"\nSentence: {s}")
    print("Tokens:", tokens)
    print("POS Tags:", tags)

# ----- HMM-based emission/transition probability calculation -----
P_book_given_VB = 0.7
P_book_given_NN = 0.3
P_start_VB      = 0.6
P_start_NN      = 0.4

score_VB = P_start_VB * P_book_given_VB
score_NN = P_start_NN * P_book_given_NN

print(f"\nScore(VB) = P(Start->VB) * P(book|VB) = {score_VB:.3f}")
print(f"Score(NN) = P(Start->NN) * P(book|NN) = {score_NN:.3f}")

best_tag = "VB" if score_VB > score_NN else "NN"
print(f"\nMost probable tag for 'book' at sentence start: {best_tag}")

```

Case Study 3: Financial News POS Tag Correction and Uncertainty Analysis

Answer 1: Applying the Transformation Rule

Initial tags: market/NN growth/NN drives/NNS investment/NN

Rule: Change NNS to VBZ if the preceding word is tagged NN.

“drives” is immediately preceded by “growth/NN”, so the rule fires on it.

Corrected tags: market/NN growth/NN drives/VBZ investment/NN

Why the transformation is linguistically appropriate: “Market growth” together forms a singular noun-phrase subject, and “drives” is the finite, present-tense main verb of the clause (meaning “causes” / “leads to”), not the plural noun “drives” as in “car drives”. Because “drives” is morphologically identical whether used as a verb or a plural noun, the tagger over-relied on the surface “-s” ending and mistagged it NNS; the transformation rule encodes the contextual regularity that a plural noun rarely follows another bare noun without an intervening verb, correcting the tag to the syntactically consistent VBZ.

Answer 2: Analysis of the “drives” Tagging Error

“drives” was initially mistagged NNS purely because it ends in “-s”, a strong statistical cue for plurality that a weakly-contextual statistical tagger over-relies on. Grammatically, however, the clause “Market growth ___ investment” has no other candidate verb, so “drives” must be the predicate verb, inflected for 3rd-person singular present tense to agree with the singular subject “market growth” — hence VBZ, not NNS.

The surrounding grammatical context is the classic Subject–Verb–Object (SVO) frame of English: a noun-phrase subject immediately before the ambiguous word and a noun-phrase object immediately after it. Only a verb can occupy that medial slot in an SVO sentence, and this is exactly the contextual regularity that transformation-based (Brill) tagging exploits to fix superficial, morphology-driven tagging mistakes that a context-blind tagger would otherwise leave uncorrected.

Answer 3: Corpus-Based Relative Frequency / Probability

Total corpus count = 500 + 350 + 180 + 420 = 1450

- $P(\text{market}) = 500 / 1450 \approx 0.3448$
- $P(\text{growth}) = 350 / 1450 \approx 0.2414$

- $P(\text{drives}) = 180 / 1450 \approx 0.1241$
- $P(\text{investment}) = 420 / 1450 \approx 0.2897$

How this supports probabilistic POS tagging: relative frequency (MLE) computed from corpus counts is the basic building block of virtually every probabilistic tagger — emission probabilities $P(\text{word}|\text{tag})$ and transition probabilities $P(\text{tag}_i|\text{tag}_{i-1})$ are themselves ratios of counts collected exactly this way from an annotated corpus. Frequent words such as “market” (about 34.5% of this local sample) get their dominant tag estimated with lower variance than rarer words, so corpus frequency also serves as an implicit confidence signal that feeds directly into entropy-based confidence measures such as those computed below.

Answer 4: Entropy of the Word-Frequency and Tag Distributions

(a) Entropy of the word-frequency distribution (using the probabilities from Answer 3):

$$H = -\sum p \log_2 p = -(0.3448 \times \log_2 0.3448 + 0.2414 \times \log_2 0.2414 + 0.1241 \times \log_2 0.1241 + 0.2897 \times \log_2 0.2897)$$

$$H \approx 0.5296 + 0.4950 + 0.3736 + 0.5178 \approx 1.916 \text{ bits}$$

(b) Entropy of the POS-tag distribution, before and after the transformation:

Before: tags are NN, NN, NNS, NN $\rightarrow P(\text{NN}) = 3/4 = 0.75$, $P(\text{NNS}) = 1/4 = 0.25$

$$H_{\text{before}} = -(0.75 \times \log_2 0.75 + 0.25 \times \log_2 0.25) = -(-0.3113 - 0.5) = 0.811 \text{ bits}$$

After: tags are NN, NN, VBZ, NN $\rightarrow P(\text{NN}) = 3/4 = 0.75$, $P(\text{VBZ}) = 1/4 = 0.25$

$$H_{\text{after}} = -(0.75 \times \log_2 0.75 + 0.25 \times \log_2 0.25) = 0.811 \text{ bits (numerically unchanged)}$$

Interpretation: The word-frequency entropy (≈ 1.916 bits) reflects the inherent uncertainty in predicting which of the four vocabulary words appears next, given their relative frequencies; it is unaffected by the tag correction because the transformation rule changes tag labels, not word counts. The POS-tag distribution entropy is 0.811 bits both before and after the transformation, because the proportion of tags (three majority-tag tokens, one minority-tag token) has the same shape in both cases — only the label of the minority tag changed from NNS to VBZ. This illustrates an important property of entropy: it measures the shape/uncertainty of a distribution, not the semantic correctness of its labels. The transformation improved linguistic accuracy without changing the statistical uncertainty of the tag distribution itself; the comparatively low value of 0.811 bits (well below the 1-bit maximum for a two-category distribution) indicates the tagging outcome for this sentence was already fairly predictable, and the rule-based correction refined which label was correct rather than how confident the pipeline could be.

Answer 5: Python Program — Transformation-Based Tagging, Frequency and Entropy

```
import nltk
from nltk import word_tokenize, pos_tag
import math

sentence = "Market growth drives investment."
tokens = word_tokenize(sentence)
print("Tokens:", tokens)

# Initial statistical POS tags, as given in the case study
initial_tags = [("Market", "NN"), ("growth", "NN"),
                ("drives", "NNS"), ("investment", "NN")]
print("Initial POS tags:", initial_tags)

# ----- Transformation-Based Tagger -----
def apply_transformation(tags):
    corrected = tags.copy()
    for i in range(1, len(tags)):
        prev_tag = corrected[i - 1][1]
        word, tag = corrected[i]
        if tag == "NNS" and prev_tag == "NN":
            corrected[i] = (word, "VBZ")
    return corrected

corrected_tags = apply_transformation(initial_tags)
print("\nInitial tags: ", initial_tags)
print("Corrected tags:", corrected_tags)

# ----- Corpus Frequency and Probability -----
freq = {"market": 500, "growth": 350, "drives": 180, "investment": 420}
total = sum(freq.values())
prob = {w: c / total for w, c in freq.items()}
print("\nRelative frequencies:")
for w, p in prob.items():
    print(f"P({w}) = {p:.4f}")

# ----- Entropy Calculations -----
def entropy(prob_dict):
    return -sum(p * math.log2(p) for p in prob_dict.values())

H_word_freq = entropy(prob)
print(f"\nEntropy of word-frequency distribution = {H_word_freq:.4f} bits")

def tag_distribution(tag_list):
    counts = {}
    for t in tag_list:
        counts[t] = counts.get(t, 0) + 1
    n = len(tag_list)
    return {t: v / n for t, v in counts.items()}

before_tags = [t for _, t in initial_tags]
after_tags = [t for _, t in corrected_tags]

H_before = entropy(tag_distribution(before_tags))
```

```
H_after = entropy(tag_distribution(after_tags))
print(f"Entropy of POS-tag distribution BEFORE transformation = {H_before:.4f} bits")
print(f"Entropy of POS-tag distribution AFTER transformation = {H_after:.4f} bits")
```